

Manual of the program `q_gen`

Lavinia Egidi
Università del Piemonte Orientale

Contents

1	Introduction	2
2	The program	3
2.1	Quick start	3
2.2	Details and options	3
2.2.1	Specifying files to process	4
2.2.2	Specifying the output directory	5
2.2.3	Specifying the configuration file	5
2.2.4	Processing	5
2.3	Concatenation	6
2.4	JSON format verification	6
3	Input file format	7
3.1	Keys for all question types	7
	"category"	7
	"question_name"	7
	"question_type"	7
	"statements"	7
	"number_of_statements"	8
	"choices"	8
3.2	Keys for "dd" questions	8
	"answers"	8
	"task"	8
	"answer_placeholder"	8
	"if_infinite"	9
3.3	Keys for "cloze" questions	9
	"cloze_type"	9
	"answers"	9
	"task"	9
3.4	Keys for "mcq" questions	9
	"group_fractions"	9
3.5	Details on the "answers" field	10

3.6	Details on the "group_fractions" field	11
3.7	Details on the "choices" field	11
3.7.1	"choices" for "mcq"	12
3.8	Options: "options" field	13
	"moodle_shuffle"	13
	"multiple_answers"	13
	"bullet_list"	13
	"permutations"	13
3.8.1	Combinations and "permutations"	14
4	Configuration file	14
4.1	Parameters useful to the user	14
	"exec"	15
	"database"	15
	"filenames"	15
4.2	Support file names	15
	"templates"	15
4.3	Translation of keys	16
	"key_names"	16
4.4	Keys and values of input files	16
	"necessary_input_fields"	16
	"meaningful_options"	17
	"supported_question_types"	17
	"cloze_types"	17
4.5	Localization	17
A	JSON file format	18
	Comments in a JSON file	18
B	config_en.json settings	19
C	Semantic checks	20
C.1	"Fatal" errors	20
C.2	Warnings	21

1 Introduction

The **q-gen** program allows generating question collections in Moodle XML format starting from input files. The input files list the alternatives to be used and other information for the preparation of the questions (question type, category, title, etc.). The produced files can be imported into Moodle.

The program is capable of producing questions of the following types:

- multiple choice
- drag&drop

- single-choice cloze
- open cloze

The program is distributed without any warranty, in the hope that it may be useful to others. It is recommended to verify that the produced questions are correct before using them, as there may be errors in the program or the meaning of some options may not be clear.

It is distributed with a few examples that can help understand the required format for the input files without needing to read this entire manual.

2 The program

2.1 Quick start

To obtain the program, open a terminal and type

```
git clone https://github.com/laviniaegidi-UPO/q_gen.git
```

A `q_gen` folder will be created in the working directory. Navigate into the `q_gen` folder. On `linux`, the command is:

```
cd q_gen
```

Rename file `config_en.json` to `config.json`. This way the output of the program will be in English. The file `config.json` distributed with the software is identical to the Italian version `config_it.json`.

In the folder, run the command:

```
python3 q_gen.py
```

This will run the program on the example files distributed with the code (located in the `source` folder) and write the output to the `collections` folder.

2.2 Details and options

The program is distributed with the following files:

- Code:
 - `q_gen.py`
 - `gen_tools.py`
 - `combinatorics.py`
 - `verify.py`
- examples:

- in **source**, examples in Italian
- in **sources**, examples in English
- configuration files:
 - **config-it.json**, configuration file to use the program in Italian
 - **config-en.json**, configuration file to use the program in English
 - **config.json**, a copy of **config-it.json**
- **"templates"** directory, with the templates for the Moodle XML formats
- **"manuals"** directory, with manuals in Italian and in English.

All code files must be in the same working directory. The configuration file **config.json** must also be present in this directory (a different configuration file can be specified, see Section 2.2.3).

The **config.json** file is a JSON format file. Appendix A briefly summarizes the syntax of this format. Section 4 details the structure of the configuration file.

The software is distributed with two configuration files, **config-it.json** e **config-en.json**. The first one defines an interface in Italian, the second in English. The default **config.json** is a copy of **config-it.json**.

The program is run from a terminal by navigating to the folder where the program is located and using the command:

```
python3 q-gen.py
```

When invoked this way, it processes the files specified in the **"exec"** list inside the **config.json** file (see Section 4). If none are specified, the program prompts for a filename or path.

2.2.1 Specifying files to process

By calling the program with the **-s/--source_dir** option, as for example in:

```
python3 q-gen.py -s my_folder/
```

the contents of the **config.json** file are ignored, and all files present in the **my_folder** directory are processed.

The program can also be called with the **-i** or **--input_file** option followed by the filename, for example:

```
python3 q-gen.py -i my-questions.json
```

In this case as well, the **"exec"** field in the configuration file is ignored.

Filenames can be provided as input:

- without prefix, without `.json` extension, and without specifying the full path (in this case, the prefix specified in the configuration file is used, and the file is searched for in the folder specified in the configuration file; see Section 4);
- with the complete path or relative path to the working directory (with or without the `.json` extension): in this case, no prefix is added, and the input directory specified in the configuration file is ignored, but the `.json` extension is appended if necessary.

2.2.2 Specifying the output directory

The XML files containing the question collections are written to the folder specified in the configuration file (by default `collections`), unless a different folder is specified on the command line using the `-o/--out_dir` option followed by the folder name. For example:

```
python3 q_gen.py -o my_collections/
```

This option can be used alongside any input option.

2.2.3 Specifying the configuration file

Finally, a different configuration file can be specified using the `-c/--config_file` option, followed by the name of the new configuration file, optionally including the path. For example:

```
python3 q_gen.py -c my_configuration.json
```

. This option can be used together with any of the previous ones.

2.2.4 Processing

At the beginning of processing, the software checks:

- that the configuration file exists,
- that all required templates exist (as specified in the configuration file),
- that each file indicated as a file to process exists,
- that the JSON format of each file to process is correct,
- that the necessary fields (defined in the configuration file) are present in each file to process,
- that each file to process is consistent (e.g., that in an MCQ question the sum of the correct options is exactly 100). See Appendix C for the complete list of these checks.

If one of these checks fails, execution terminates with an error message or, in some cases, only a warning is issued and processing continues because the issue is non-blocking.

After the checks, the combinations are generated and the question collection is produced. The collection is saved in the folder specified in the configuration file or on command line, and the prefix defined in the configuration file is added to the filename. If no output folder is specified, the file is saved in the current folder.

2.3 Concatenation

The program produces an XML file for each input file. It is useful to import them into Moodle one by one to verify the correctness of the questions. However, to speed up the import process, you can use the concatenation option provided by the `q-gen.py` program.

With the `-j` or `--join_dir` option, the program is instructed to concatenate all question collections located in the specified folder:

```
python3 q-gen.py -j my_collections/
```

The output file is named `000_all.xml`¹ and is generated in the same folder (`my_collections` in the example).

Warning: the collections to be concatenated must have already been generated.

If the `000_all.xml` file is already in the `my_collections` folder, it will not be concatenated with the other files and will be overwritten, unless its name has been changed (or the name definition in the configuration file has been modified; see Section 4 regarding this).

When called with option `-j` or `--join_dir`, the program performs the concatenation and terminates.

2.4 JSON format verification

Since it is possible to create or modify the configuration file and the program does not normally verify the JSON format of the configuration file, the program provides a specific option for this check.

Calling it with the `-v` or `--verify_json_file` option followed by a filename, the program verifies the JSON format of the file:

```
python3 q-gen.py -v file.json.
```

A valid file `config.json` must be regularly present in the working directory.

When called with option `-v` or `--verify_json_file`, the program performs the verification and terminates.

¹Default name defined in `config-en.json`, that can be customized.

3 Input file format

Input files must be in JSON format (see Appendix A). In this document, the key names defined in file `config_en.json` in the distribution are used.

3.1 Keys for all question types

`"category"`

Mandatory. The category in Moodle where questions should be placed.

`"question_name"`

Mandatory. The name of the questions; questions coming from a single input file will share the same name.

`"question_type"`

Mandatory. The question type. The following values can be specified for the `"question_type"` key:

- `"cloze"`: cloze type
- `"dd"`: drag & drop type
- `"mcq"`: multiple choice question type.

`"statements"`

Mandatory. The statements or, in the case of MCQ, the question and the possible answers.

The value of the `"statements"` key is a dictionary for `"dd"` and `"cloze"` questions and has keys `"1"`, `"2"`, `"3"`, ... (quoted consecutive numbers starting from 1). Each number identifies a group of statements, and the key value is a list. Each statement is in turn a dictionary with keys `"statement"` and `"correct"`. Therefore the `"statements"` key/value pair could look for instance like this:

```
"statements"{
  "1": [{"statement": "statement1", "correct": "answer1"},
        {"statement": "statement2", "correct": "answer2"}],
  "2": [{"statement": "statement3", "correct": "answer3"}]
}
```

The value of the `"correct"` key specifies the correct answer; this is explained in `"answers"` in Section 3.5.

For "mcq" questions, each statement is not a dictionary but simply a string. The first group is assumed to contain the set of possible questions. All of the questions must share the same correct answer(s). So for instance:

```
"statements"{
  "1":["Which is the color of the sun?",
      "Which is the color of a lemon?"],
  "2":["yellow"],
  "3":["blue","purple","black"]
}
```

In this case, the correct answer is specified via the value of the "group_fractions" key. See Section 3.4 and Section 3.6.

"number_of_statements"

Mandatory. The number of statements per question to be chosen from "statements". In the case of "mcq", "number_of_statements" is one plus the number of possible answers to choose from. This is because the question in an "mcq" is also randomly selected from a group of questions—those in group "1".

"choices"

Mandatory for "mcq" questions; optional for other question types. See Section 3.7

3.2 Keys for "dd" questions

In addition to the keys in Section 3.1, the following keys are mandatory for "dd" questions:

"answers"

Mandatory. These are the choices that can be moved (drag&drop) to answer. See Section 3.5 for details.

"task"

Mandatory. The instructions, such as, e.g., “complete the following statements”.

"answer_placeholder"

Mandatory. The string to be written in place of the gap to fill; it must not appear elsewhere in the JSON file.

"if_infinite"

Optional. Specifies whether the drag&drop choices run out or remain available infinitely; if omitted, choices are assumed to be exhaustible.

3.3 Keys for "cloze" questions

In addition to the keys in Section 3.1, the following fields are mandatory for "cloze" questions:

"cloze_type"

Mandatory. Currently, the value can be:

- "MULTICHOICE_H",
- "MULTICHOICE_V",
- "MULTICHOICE_VS",
- "MULTICHOICE_HS",
- "SHORTANSWER"

"answers"

Mandatory. Specifies the available choices or accepted answers. See Section 3.5 for details.

"task"

Mandatory. The instructions, such as “choose the correct option”, “complete the following statements”,...

3.4 Keys for "mcq" questions

In addition to the tags in Section 3.1, the following fields are mandatory for "mcq" questions:

"group_fractions"

Mandatory. The value is a dictionary specifying the grade value for each group of "statements": 100, 0, or a fraction as used by Moodle. See Section 3.6 for details.

3.5 Details on the "answers" field

The "answers" field is always a dictionary. There are some constraints on the keys:

- for "MULTICHOICE" cloze questions, the keys can be arbitrary; so for example

```
"answers": {"4": "red", "AA": "green", "5": "yellow"}
```

if you want the three options to be red, green, and yellow

- for SHORTANSWER cloze questions, there must be one key per "statements" group, i.e., "1", "2", "3", ...; the associated value must follow the exact format required for a SHORTANSWER cloze. For example:

```
"answers": {  
  "1": "{1:SHORTANSWER:=red}",  
  "2": "{1:SHORTANSWER:=yellow}",  
  "3": "{1:SHORTANSWER:=red}"  
}
```

- for drag&drop questions, the keys must be consecutive numbers starting from "1". Example:

```
"answers": {"1": "red", "2": "green", "3": "blue", "4": "yellow"}
```

The "correct" field in the dictionaries representing the statements (Section 3.1) contains the key of "answers" whose value is the correct answer for that sentence. Therefore, if

```
"answers" : {"1": "red", "2": "green", "3": "blue", "4": "yellow"}
```

and

```
"statements": {  
  "1": [{"statement": "statement1", "correct": "2"},  
        {"statement": "statement2", "correct": "4"}],  
  "2": [{"statement": "statement3", "correct": "1"}]  
}
```

the correct answer for statement1 is green, for statement2 is yellow, and for statement3 is red.

In the case of a **SHORTANSWER**, for convenience, each group can be a list, but in that case, every statement in each group must share the same **SHORTANSWER**; the key of the corresponding **SHORTANSWER** is the group key. For example, you can write:

```
"statements":{
  "1":["statement1","statement2"],
  "2":["statement3"]
}
```

which is interpreted by the program as

```
"statements":{
  "1":[{"statement":"statement1", "correct":"1"},
      {"statement":"statement2", "correct":"1"}],
  "2":[{"statement":"statement3", "correct":"2"}]
}
```

3.6 Details on the "group_fractions" field

The "group_fractions" field is a dictionary. Here, for each group, the answer score is specified, except for group "1" which contains the possible questions. The values, as required by Moodle, are for example "100", "50", "0", "-50", "-3,3333" (always written in quotes, both period and comma can be used interchangeably as decimal separators). Example:

```
"group_fractions":{"2":"100","3":"0","4":"50","6":"-50"}
```

The fact that key "5" is missing implies "5":"0". All sentences within each group therefore share the same fraction value, and as a result, statements must be grouped based on their truth value (true together, false together). It is the author's responsibility to select sentences in a way that makes sense, but the program performs some basic checks. Further details are in Section 3.7.1 (but you probably need to read all of Section 3.7) and Appendix C.

3.7 Details on the "choices" field

The "choices" field, if present, is a list of lists and specifies how statements are selected from the various groups in "statements".

For example:

```
"choices":[[1,2,1],[1,1,1,1]]
```

Each nested list must have a length at most equal to the number of groups. If there are 3 groups, [1,2,1] means that the program must select 1 statement from group "1", 2 from "2", and 1 from "3". If there are, for instance, 4 groups, [1,2,1] is equivalent to [1,2,1,0].

If there are multiple lists in "choices", combinations are chosen as specified for each list.

For each list contained in "choices", the sum of the values must equal "number_of_statements". Therefore, if "number_of_statements": "4" and there are at least 5 groups, the list [1, 2, 1, 0, 0] is valid, but the list [2, 1, 0, 2, 2] is not, because its elements sum to 7.

If the "choices" field is missing or empty ("choices": []), the program chooses all possible combinations: be aware that these could be many, potentially too many.

3.7.1 "choices" for "mcq"

In the case of "mcq", the "choices" field is mandatory and each list must start with 1 (for example [1,0,2,1], but not [2,0,1,1]). Recall that the first group in the "statements" is interpreted as a list of possible questions (see "statements" in Section 3.1) and exactly one question must always be selected. Of course this only makes sense if all questions listed in group "1" share the same correct answer.

To complete the earlier example,

```
"number_of_statements":4,
"choices":[1,1,2],
"group_fractions":{"2":"100","3":"0"},
"statements":{"
  "1":["Which is the color of the sun?",
      "Which is the color of a lemon"],
  "2":["yellow"],
  "3":["blue","purple","black"]}
}
```

If the list specifies an answer with "group_fraction" 100, the program automatically treats it as an MCQ with a single correct answer. Otherwise, an MCQ with multiple correct answers is created. A question with an answer fraction of 100% can be forced to accept multiple responses using the "multiple_answers" option (see options in Section 3.8).

For an "mcq", the software also verifies, for each list in "choices", that the sum of positive "group_fractions" values equals exactly 100. For example, this is valid:

```
"choices":[[1,1,2],[1,1,1,1]]
```

```
"group_fractions":{"2":"100","3":"0","4":"0"}
```

whereas, in the following example, none of the "choices" are acceptable:

```
"choices":[[1,2,1],[1,0,2,1]]  
"group_fractions":{"2":"100","3":"0","4":"50"}
```

Additionally, for a question allowing multiple responses, the program checks that no answers have a fraction of 0; in this case, it only raises a warning and continues processing. Thus, in this scenario, a warning will be issued for the second list but not for the first:

```
"choices":[[1,1,2],[1,0,1,2]]  
"group_fractions":{"2":"100","3":"0","4":"50"}
```

The field "choices" is mandatory for "mcq" questions because the user must keep control of the choices.

3.8 Options: "options" field

The "options" field is optional. It is a list. The possible options are:

"moodle_shuffle"

Instructs the program to set "Shuffle the choices" for these questions. It makes sense for "mcq", "dd", and "cloze" of type "MULTICHOICE".

"multiple_answers"

Makes sense for "mcq" questions and instructs the program to define all questions as multiple-response questions, even those having a single correct answer with a "100" fraction.

"bullet_list"

Presents the items as a bulleted list. It makes sense for "dd" and "cloze".

"permutations"

Requests writing the combinations in all possible orders; to clarify this well, we go into detail in Section 3.8.1.

3.8.1 Combinations and "permutations"

The program calculates all possible combinations of sentences according to the requirements specified in **"choices"**, but without permutations; that is, if sentence combination A,B,C is chosen, A,C,B will not be chosen, nor B,C,A, nor B,A,C, etc. When writing the corresponding question to the output collection, the three sentences A,B and C will not necessarily appear in this order; the program selects only one of the permutations at random and it might choose, say, C,B,A, but only that one.

If, on the other hand, the **"permutations"** option is used, the program will produce a question for each permutation. In our example, 6 questions with the same sentences but in a different order: A,B,C and A,C,B and B,A,C and B,C,A and C,A,B and C,B,A. It essentially simulates the Moodle "Shuffle" option of an MCQ question.

It makes sense for **"dd"** and **"cloze"** questions.

Note that the **"moodle_shuffle"** option reorders the drag&drop choices for **"dd"** questions, not the sentences. For **"MULTICHOICE"** **"cloze"** questions, the versions with the **S** at the end (**"MULTICHOICE_VS"** and **"MULTICHOICE_HS"**) similarly reorder the choices for the answers.

4 Configuration file

In the folder where the program is located (and executed), the configuration file containing the definition of the parameters necessary for running the program must be present. By default, a configuration file **config.json** is expected. As discussed in Section 2.2.3, a different configuration file can be specified.

The software is distributed with two configuration files, **config_it.json** and **config_en.json**. In the distribution there is also a default **config.json** which is actually **config_it.json**.

In this manual in English, examples are taken from the file **config_en.json**, but the structure of the configuration files is common. If you decide to use a different configuration file, make sure that the new file contains all the necessary definitions.

The keys in the configuration file can never be modified (so they are the same in **config_en.json** and **config_it.json**); otherwise, it would be necessary to modify the program's code. Conversely, some of the values of these keys can be customized, as specified below.

4.1 Parameters useful to the user

The values of the keys described in this section can always be safely modified, ensuring consistency with files and directories.

"exec"

It is a list of file names, which may be empty, i.e., []; if there are strings, they are interpreted as file names on which to run the program. If the program is launched with input options (see Section 2.2.1), "exec" is ignored.

"database"

It is a list of file names; if "ALL" is written in "exec", the program is executed on all the files listed in "database".

"filenames"

It is a dictionary containing directory names and file prefixes. In particular

- "source_dir" and "out_dir" specify the default folders (contained in the same folder as the program) from which the program must read the input files and into which it must write the output files, respectively;
- "sourcefile_prefix" and "outfile_prefix" specify the prefix that each input and output file must have, respectively. For example, it may be convenient not to use separate folders for input and output, but instead distinguish input and output files by their prefix, such as `source_colors.json` and `collection_colors.xml`.
- "collections_concat" defines the name of the output file of the program when used with the `-c/--concat` option (see Section 2.3).

All values of these keys, except for "collections_concat", can be set to "", i.e., left empty.

The values of the keys in the "filenames" dictionary, in the `config.json` file distributed with the software, are listed in table 1 in appendix B.

As explained in section 2.2.1, the program is very flexible regarding how input file names are written in "exec" and "database".

4.2 Support file names

You can change the folder name, but not all file names, due to how they are used by the program. Since customizing them is not very useful, it is best to leave them as they are. The set names are listed in Table 3 in Appendix B.

"templates"

It is a dictionary and defines the folder and file names in which the templates used to compose question files in Moodle XML format are defined.

4.3 Translation of keys

The `"key_names"` dictionary was introduced to allow customizing key names in the input JSON files. Keys cannot be modified, only values. This dictionary is used by the program to correctly read the input files.

`"key_names"`

It is a dictionary. By modifying the values associated with individual keys, you can change the key names required in the JSON file, for example to write them in a different language.

If you want to change the name of a field in the input files (e.g., `"svar"` instead of `"answers"`), you can change the value next to `"question_type"`, i.e., `"question_type":"svar"`. Obviously, you must then write `"svar"` instead of `"answers"` in all input files.

You cannot change any dictionary keys.

The dictionaries described in Section 4.4 should be read by translating the values with the `"key_names"` dictionary: for example, if `"answers"` appears in one of the lists of the `"necessary_input_fields"` dictionary, and in the dictionary the value of `"answers"` is `"svar"`, namely

```
"answers" : "svar",
```

the required field is `"svar"`.

Notice that in the definition of the keys in `config.en.json`, keys and values are the same (`"answers":"answers"`) and this is why the example above is not taken from the `config.en.json` file.

The values set in the `config.en.json` file distributed with the software are listed in Table 2 in Appendix B.

4.4 Keys and values of input files

The following dictionaries define, together with the `"key_names"` dictionary, the keys and values that the program expects for input JSON files. Modifications to these lists can cause errors or inconsistencies. The fields should be read after translation via the `"key_names"` dictionary, as explained at the end of Section 4.3.

`"necessary_input_fields"`

It is a dictionary. It lists the necessary fields in the input JSON files, divided into various lists: `"all"` applies to all question types, while `"dd"`, `"mcq"`, and `"cloze"` apply to individual types. This dictionary is used to check the consistency of the file to be processed.

"meaningful_options"

It is a dictionary. It is used to verify that the options in the input file exist and are consistent with the question type.

"supported_question_types"

Lists the question types (currently "dd", "mcq", and "cloze") handled by the program. It cannot be modified.

"cloze_types"

Lists the cloze types handled by the program. It cannot be modified.

4.5 Localization

The program is designed to make it simple to change the interface language, except for the help obtained with the command

```
python3 q_gen.py --help
```

which always responds in English.

To translate it, simply translate the messages collected in the "messages" dictionary within the configuration file. The messages in the dictionary are divided into information, error, and warning messages. Furthermore, they are handled differently depending on the group or subgroup (which is defined by how the message key begins). The groups are:

- "errors": the key for these messages always starts with "ERROR_", they are displayed with a red x next to them, and execution terminates immediately after the message is shown;
- "warnings": the key for these messages always starts with "WARNING_", they are displayed with a yellow triangular icon next to them, and execution continues after the message;
- "infos": these messages are divided into subcategories, but in all cases execution continues after printing the message:
 - those whose key starts with "MSG_" are written without an icon
 - those whose key starts with "OK_" are written with a green checkmark icon next to them
 - those whose key starts with "INFO_" are written with a gray arrow icon next to them
 - those whose key starts with "DET_" are written with a magnifying glass icon next to them.

When translating messages, variables, that is words preceded by “\$”, must be left unchanged, as they will be replaced by the program with the correct parameter.

A JSON file format

A JSON file is a *dictionary*, meaning it has this format:

```
{  
  "key1":value1,  
  "key2":value2,  
  ...  
  "keyN":valueN  
}
```

In particular, the file contents are enclosed in curly braces and the file is a sequence of *key-value pairs*, separated by commas. The last pair does not end with a comma.

Every *key* is a string (a word in double or single quotes). Every *value* can be a string (e.g., "ciao"), another dictionary, or a *list* (see below).

If, for example, the *key-value* pair (or *item*) of a dictionary is

```
"color":"red"
```

we say that "red" is *the value of the key* "color".

A *list* is a sequence of values enclosed in square brackets, separated by commas:

```
[value1, value2,...,valueN]
```

(the last value is not followed by a comma). The elements of a list can themselves be dictionaries, lists, strings, or numbers.

Comments in a JSON file

There is no specific format for adding comments in a JSON file.

However, you can add any key you like to a JSON file. In our case, if the key is not specified in this manual, it is simply ignored by the **q-gen** program.

Therefore, to insert a comment in a JSON file, simply add a new key-value pair to the dictionary using a new key. For example:

```
"INFO":"Group '2' is the most important."
```

The "INFO" field will be ignored.

Note that in the value of "INFO", single quotes are used around 2 instead of double quotes. Double quotes would prematurely terminate the value of the "INFO" key.

B config-en.json settings

The following tables summarize the settings of the `config-en.json` file distributed with the software.

Table 1: List of user file names in file `config-en.json` (customizable)

Keys in <code>config-en.json</code> , cannot be changed	Current file names, customizable
"source_dir"	"source"
"out_dir"	"collections"
"sourcefile_prefix"	""
"outfile_prefix"	"collection_"
"collections_concat"	"000_all"

Table 2: List of keys and options in file `config-en.json` (customizable)

Keys in <code>config-en.json</code> , cannot be changed	Current values, customizable
"category"	"category"
"question_name"	"question_name"
"question_type"	"question_type"
"number_of_statements"	"number_of_statements"
"statements"	"statements"
"answers"	"answers"
"task"	"task"
"answers_placeholder"	"answer_placeholder"
"cloze_type"	"cloze_type"
"group_fractions"	"group_fractions"
"choices"	"choices"
"computed_choices"	"computed_choices"
"options"	"options"
"statement"	"statement"
"correct"	"correct"

Continued on next page

Table 2: List of keys and options in file `config-en.json` (customizable)
(Continued)

Keys in <code>config-en.json</code> , cannot be changed	Current values, customizable
"if_infinite"	"if_infinite"
"multiple"	"multiple_answers"
"moodle_shuffle"	"moodle_shuffle"
"itemized"	"bullet_list"
"all_orders"	"permutations"

Table 3: List of the names of the template files in file `config-en.json` (cannot be changed)

Keys in file <code>config-en.json</code> , cannot be changed	File names, cannot be changed
"template_dir"	"templates"
"template_collection"	"shell"
"template_dragbox"	"dragbox"
"template_MCQ_answers"	"mcq_answers"
"template_dd"	"dd"
"template_mcq"	"mcq"
"template_cloze"	"cloze"

C Semantic checks

At the beginning of execution, after verifying that the input file is in the correct JSON format, the software performs some checks that make sense within the logic of this specific software (which are therefore called *semantic*, as opposed to the syntactic check that verifies the JSON format).

C.1 “Fatal” errors

Some checks verify essential aspects for the software’s operation. Therefore, if they fail, execution stops and an error message is displayed.

These checks are as follows:

- all keys that are mandatory for all types of questions are present (as specified in the configuration file)

- the question type is one of the three allowed ("dd", "cloze", or "mcq")
- all keys that are mandatory for the specific question type are present, depending on the question type (as listed in the configuration file)
- if the question is a "cloze", the "cloze_type" is in the list that appears in the configuration file
- the value of the "statements" key is a dictionary
- the groups (keys of the "statements" dictionary) are numbered from 1 onwards, with no missing numbers
- for each statement, if it is a dictionary, it contains the key "statement" and the key "correct"
- in the case of "dd", the placeholder string that is specified by the value of the key "answer_placeholder" appears in every statement
- each list in "choices" has a length at most equal to the number of groups (i.e., keys) in "statements"
- the values in each list of "choices" sum up to "number_of_statements"
- each value in each list of "choices" is not greater than the number of statements in the corresponding group (i.e., if the list specifies 3 for a group, there must be at least 3 statements to choose from in that group)
- if the question is "mcq", the first value of each choice must be 1
- if the question is "mcq", the sum of the fractions of choices with a positive fraction must be 100.

C.2 Warnings

Other checks generate a warning, but computation proceeds. These are:

- the question is an "mcq" that accepts multiple answers, but there are statements with a zero score
- a list in "choices" has a length less than the number of groups; in this case, the software warns that a value of 0 will be considered for the remaining groups
- the question is "dd" and for some drag&drop answers it is not specified whether they should be finite or infinite; the software warns that it will consider them finite
- an option is specified that makes no sense for that question type (according to what is defined in the configuration file); the software warns that this option will be ignored.